

LAPORAN TUGAS SISTEM OPERASI (PRAKTIKUM)
TUGAS P2: PERINTAH DASAR LINUX
CHAPTER 4 DAN CHAPTER 5

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Sistem Operasi



Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	4
1.1 Lingkungan Pengerjaan	4
1.2 Cara Pengerjaan	4
BAB II CHAPTER 4: SHELL FEATURES	5
2.1 Soal 1: Menampilkan teks dengan quoting yang tepat	5
2.2 Soal 2: Menampilkan isi variabel	5
2.3 Soal 3: Quoting ANSI-C	6
2.4 Soal 4: Nama berkas diawali angka	6
2.5 Soal 5: Ekstensi tidak diawali huruf t atau l	7
2.6 Soal 6: Ekstensi satu karakter	7
2.7 Soal 7: Ekstensi selain txt	8
2.8 Soal 8: Membaca pola wildcard	8
2.9 Soal 9: Nama hanya huruf kecil sebelum ekstensi	9
2.10 Soal 10: Nama diawali ma, he, atau hi	9
2.11 Soal 11: Menelusuri sub-direktori sampai ke dalam	10
2.12 Soal 12: Brace expansion untuk membuat dan menyalin berkas	11
2.13 Soal 13: Perintah bawaan set	11
2.14 Soal 14: Operator pipe dan perintah tee	12
2.15 Soal 15: printf memakai ulang format	12
2.16 Soal 16: Membuat berkas dengan brace expansion	13
2.17 Soal 17: Menambahkan baris kombinasi ke berkas	14
2.18 Soal 18: Perbedaan operator < dan 	15
2.19 Soal 19: Karakter untuk mewakili stdin	15
2.20 Soal 20: Operator pengalihan keluaran	16
2.21 Soal 21: Masukan dari keyboard sampai Ctrl+D	17
2.22 Soal 22: Here document	18
2.23 Soal 23: Here string dan interpolasi variabel	18

2.24 Soal 24: Mengalihkan keluaran beberapa perintah sekaligus	19
2.25 Soal 25: Operator && dan 	19
2.26 Soal 26: Menjaga baris baru hasil command substitution	20
2.27 Soal 27: Process substitution dan brace expansion	20

BAB III CHAPTER 5: VIEWING PART OR WHOLE FILE CONTENTS 21

3.1 Soal 1: Memberi nomor baris tanpa baris kosong berulang	21
3.2 Soal 2: Menggabungkan stdin dengan isi berkas	21
3.3 Soal 3: Membalik urutan baris gabungan atau per berkas	22
3.4 Soal 4: Membalik blok teks dengan pemisah khusus	23
3.5 Soal 5: Opsi -n dan -N pada less	23
3.6 Soal 6: Membuka berkas lain dari dalam less	24
3.7 Soal 7: Mengambil tiga karakter pertama ke variabel	24
3.8 Soal 8: Menampilkan semua baris kecuali baris pertama	25
3.9 Soal 9: Dua baris pertama dari beberapa berkas	25
3.10 Soal 10: Mengambil karakter ke-11 sampai ke-14	26
3.11 Soal 11: Enam byte pertama dari dua berkas	26
3.12 Soal 12: Enam byte terakhir dari dua berkas	27

BAB IV PENUTUP 28

4.1 Catatan Hasil	28
4.2 Kesimpulan	28

BAB I PENDAHULUAN

Tugas P2 melanjutkan Tugas P1 dengan bahan chapter 4 dan chapter 5 buku **Computing from the Command Line** karya Sundeep Agarwal. Chapter 4 membahas Shell Features, yaitu quoting, wildcard, brace expansion, pengalihan masukan dan keluaran, serta pipeline. Chapter 5 membahas Viewing Part or Whole File Contents, yaitu perintah `cat`, `tac`, `less`, `head`, dan `tail`. Laporan ini memuat jawaban 27 soal chapter 4 dan 12 soal chapter 5, masing-masing beserta tangkapan layar hasil percobaannya.

1.1 Lingkungan Pengerjaan

Buku ini memakai GNU coreutils, sedangkan macOS memakai userland BSD yang beberapa opsinya berbeda arti, jadi seluruh percobaan dijalankan di Linux seperti pada Tugas P1. Mesin yang dipakai adalah Ubuntu 24.04 yang berjalan di OrbStack pada MacBook.

Tabel 1.1: Versi perangkat lunak yang dipakai

Komponen	Versi
Sistem operasi	Ubuntu 24.04.5 LTS
Kernel	Linux 6.19.13-orbstack aarch64
Shell	GNU bash 5.2.21
Coreutils (<code>ls</code> , <code>cat</code> , <code>head</code> , <code>tail</code> , <code>tac</code>)	GNU coreutils 9.4
<code>grep</code>	GNU grep 3.11
<code>sed</code>	GNU sed 4.9
<code>less</code>	less 590
<code>man</code>	man-db 2.12.0

Paket `man-db` sengaja dipasang karena image Ubuntu bawaan OrbStack tidak menyertakan perintah `man`, padahal soal 5 dan 6 chapter 5 meminta pembacaan manual perintah `less`.

1.2 Cara Pengerjaan

1. Tiap soal chapter 4 dikerjakan di folder percobaannya sendiri, yaitu `~/tugas2-so/ch4-NN`, supaya berkas dari satu soal tidak mengganggu soal lain.
2. Berkas contoh untuk soal wildcard dibuat oleh skrip `globs.sh` dan `ls.sh` bawaan buku yang ada di `~/cli-computing/example_files/scripts/`.
3. Soal chapter 5 dikerjakan di `~/cli-computing/example_files/text_files/` karena memakai berkas contoh dari folder tersebut.
4. Tangkapan layar diambil dari jendela Terminal yang benar-benar menjalankan perintahnya, bukan hasil salin tempel. Judul jendela memperlihatkan `orb -m ubuntu` dan prompt memperlihatkan `ghaisan@ubuntu` beserta folder kerjanya.

BAB II CHAPTER 4: SHELL FEATURES

Bagian ini berisi 27 soal chapter 4. Beberapa soal memakai extended glob dan globstar yang harus dinyalakan dulu lewat `shopt`, karena keduanya tidak aktif secara bawaan.

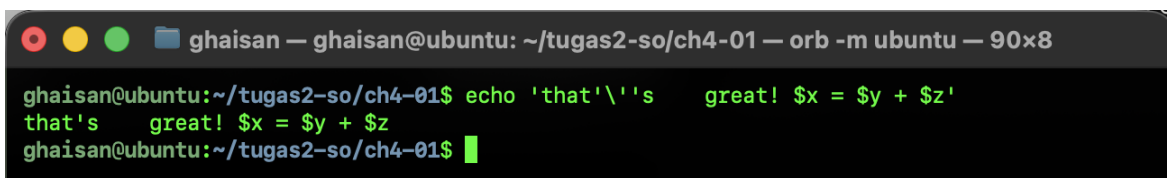
2.1 Soal 1: Menampilkan teks dengan quoting yang tepat

Soal. Tampilkan teks `that's great! $x = $y + $z` memakai `echo` dan quoting yang sesuai.

Jawaban.

```
echo 'that\'\'s great! $x = $y + $z'
```

Petik tunggal membuat seluruh isinya dianggap teks biasa, jadi `$x` tidak diganti nilai variabel dan spasi berderet tetap utuh. Tanda petik pada kata `that's` tidak bisa ditulis langsung di dalam petik tunggal, jadi petiknya ditutup dulu, ditulis `\'`, lalu dibuka lagi. Cara lain yang hasilnya sama: `echo "that's" ' great! $x = $y + $z'.`



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-01 — orb -m ubuntu — 90x8
ghaisan@ubuntu:~/tugas2-so/ch4-01$ echo 'that\'\'s great! $x = $y + $z'
that's great! $x = $y + $z
ghaisan@ubuntu:~/tugas2-so/ch4-01$
```

Gambar 2.1: hasil percobaan soal 1

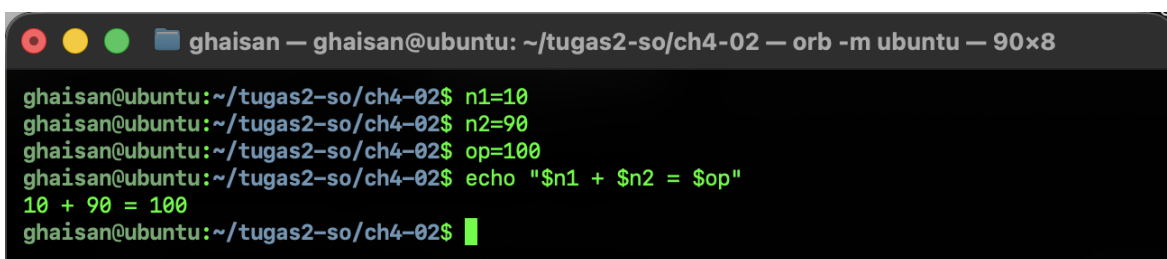
2.2 Soal 2: Menampilkan isi variabel

Soal. Tampilkan nilai variabel `n1`, `n2`, dan `op` dengan format `10 + 90 = 100`.

Jawaban.

```
n1=10
n2=90
op=100
echo "$n1 + $n2 = $op"
```

Petik ganda tetap mengganti `$nama` dengan isi variabelnya sekaligus menjaga spasi di antara teks. Kalau memakai petik tunggal, yang tercetak justru tulisan `$n1 + $n2 = $op` apa adanya.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-02 — orb -m ubuntu — 90x8
ghaisan@ubuntu:~/tugas2-so/ch4-02$ n1=10
ghaisan@ubuntu:~/tugas2-so/ch4-02$ n2=90
ghaisan@ubuntu:~/tugas2-so/ch4-02$ op=100
ghaisan@ubuntu:~/tugas2-so/ch4-02$ echo "$n1 + $n2 = $op"
10 + 90 = 100
ghaisan@ubuntu:~/tugas2-so/ch4-02$
```

Gambar 2.2: hasil percobaan soal 2

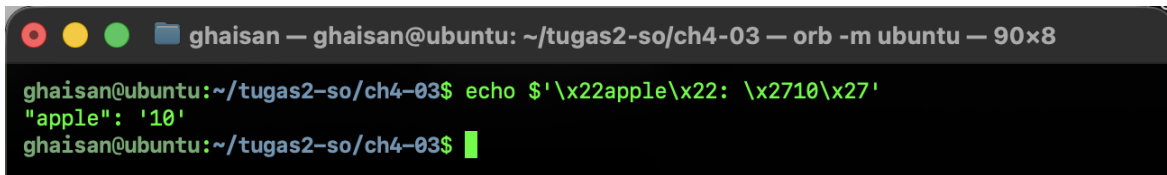
2.3 Soal 3: Quoting ANSI-C

Soal. Apa keluaran perintah `echo '$'\x22apple\x22: \x2710\x27'`?

Jawaban.

```
echo '$'\x22apple\x22: \x2710\x27'
```

Bentuk `'...'` membuat bash menerjemahkan escape seperti `\xNN` sebagai kode heksadesimal sebuah karakter. `\x22` adalah petik ganda dan `\x27` adalah petik tunggal, sehingga keluarannya `"apple": '10'`.



```
ghaisan@ubuntu:~/tugas2-so/ch4-03$ echo '$'\x22apple\x22: \x2710\x27'
"apple": '10'
ghaisan@ubuntu:~/tugas2-so/ch4-03$
```

Gambar 2.3: hasil percobaan soal 3

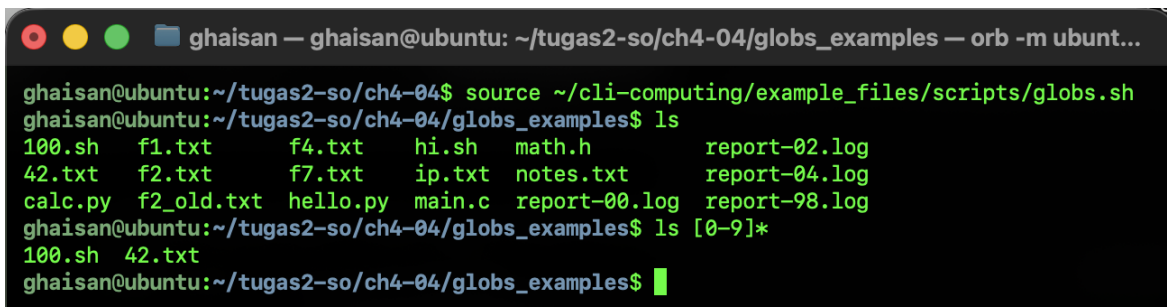
2.4 Soal 4: Nama berkas diawali angka

Soal. Tampilkan nama berkas yang diawali karakter digit.

Jawaban.

```
source ~/cli-computing/example_files/scripts/globs.sh
ls [0-9]*
```

Kurung siku cocok dengan satu karakter dari daftar di dalamnya, jadi `[0-9]` berarti satu digit di posisi pertama, lalu `*` membebaskan sisa namanya. Berkas contoh dibuat oleh skrip `globs.sh` bawaan buku.



```
ghaisan@ubuntu:~/tugas2-so/ch4-04$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-04/globs_examples$ ls
100.sh  f1.txt  f4.txt  hi.sh  math.h  report-02.log
42.txt  f2.txt  f7.txt  ip.txt  notes.txt  report-04.log
calc.py f2_old.txt hello.py main.c  report-00.log  report-98.log
ghaisan@ubuntu:~/tugas2-so/ch4-04/globs_examples$ ls [0-9]*
100.sh 42.txt
ghaisan@ubuntu:~/tugas2-so/ch4-04/globs_examples$
```

Gambar 2.4: hasil percobaan soal 4

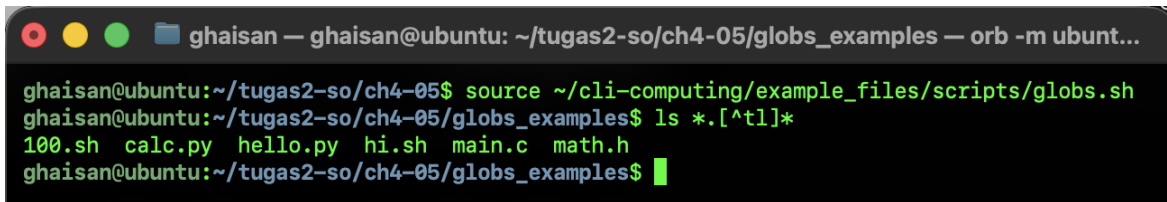
2.5 Soal 5: Ekstensi tidak diawali huruf t atau l

Soal. Tampilkan nama berkas yang ekstensinya tidak diawali `t` atau `l`, dengan asumsi ekstensi minimal satu karakter.

Jawaban.

```
ls *.[^tl]*
```

`[^tl]` cocok dengan satu karakter selain `t` dan `l`, dan posisinya tepat sesudah titik. Bash juga menerima bentuk `[!tl]` dengan arti yang sama.



```
ghaisan@ubuntu: ~/tugas2-so/ch4-05/globs_examples — orb -m ubuntu...
ghaisan@ubuntu:~/tugas2-so/ch4-05$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-05/globs_examples$ ls *.[^tl]*
100.sh  calc.py  hello.py  hi.sh  main.c  math.h
ghaisan@ubuntu:~/tugas2-so/ch4-05/globs_examples$
```

Gambar 2.5: hasil percobaan soal 5

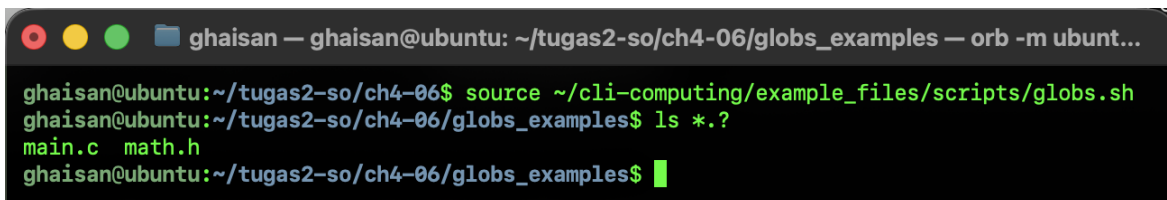
2.6 Soal 6: Ekstensi satu karakter

Soal. Tampilkan nama berkas yang ekstensinya hanya satu karakter.

Jawaban.

```
ls *.*?
```

`?` cocok dengan tepat satu karakter. Karena polanya berhenti sesudah `?`, hanya berkas dengan ekstensi satu huruf seperti `main.c` dan `math.h` yang cocok.



```
ghaisan@ubuntu: ~/tugas2-so/ch4-06/globs_examples — orb -m ubuntu...
ghaisan@ubuntu:~/tugas2-so/ch4-06$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-06/globs_examples$ ls *.*?
main.c  math.h
ghaisan@ubuntu:~/tugas2-so/ch4-06/globs_examples$
```

Gambar 2.6: hasil percobaan soal 6

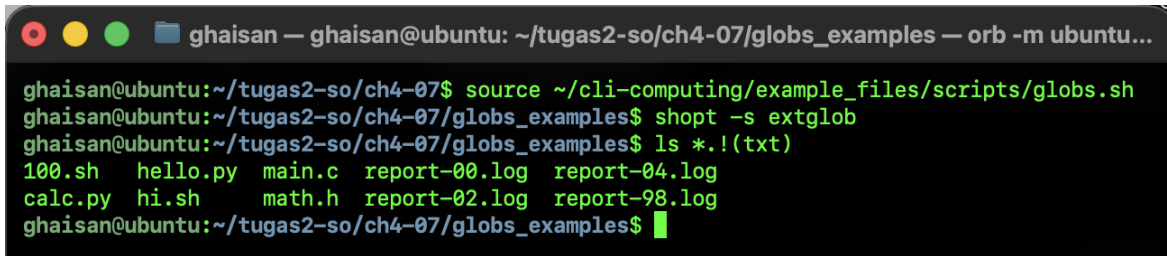
2.7 Soal 7: Ekstensi selain txt

Soal. Tampilkan nama berkas yang ekstensinya bukan `txt`.

Jawaban.

```
shopt -s extglob
ls *!(txt)
```

`!(pola)` adalah extended glob yang cocok dengan apa saja kecuali polanya. Fitur ini mati secara bawaan pada bash non-interaktif, jadi dinyalakan dulu dengan `shopt -s extglob`.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-07/globs_examples — orb -m ubuntu...

ghaisan@ubuntu:~/tugas2-so/ch4-07$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-07/globs_examples$ shopt -s extglob
ghaisan@ubuntu:~/tugas2-so/ch4-07/globs_examples$ ls *!(txt)
100.sh  hello.py  main.c  report-00.log  report-04.log
calc.py hi.sh     math.h  report-02.log  report-98.log
ghaisan@ubuntu:~/tugas2-so/ch4-07/globs_examples$
```

Gambar 2.7: hasil percobaan soal 7

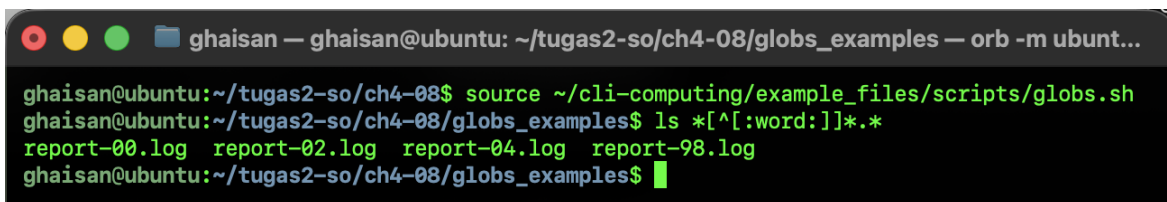
2.8 Soal 8: Membaca pola wildcard

Soal. Jelaskan pola wildcard pada perintah `ls *[^[:word:]]*.*`.

Jawaban.

```
ls *[^[:word:]]*.*
```

Pola ini mencari nama berkas yang memuat minimal satu karakter bukan word character, yaitu bukan huruf, angka, atau garis bawah, sebelum titik ekstensi. Di folder contoh yang memenuhi hanya berkas `report-*.log` karena namanya mengandung tanda hubung.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-08/globs_examples — orb -m ubuntu...

ghaisan@ubuntu:~/tugas2-so/ch4-08$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-08/globs_examples$ ls *[^[:word:]]*.*
report-00.log  report-02.log  report-04.log  report-98.log
ghaisan@ubuntu:~/tugas2-so/ch4-08/globs_examples$
```

Gambar 2.8: hasil percobaan soal 8

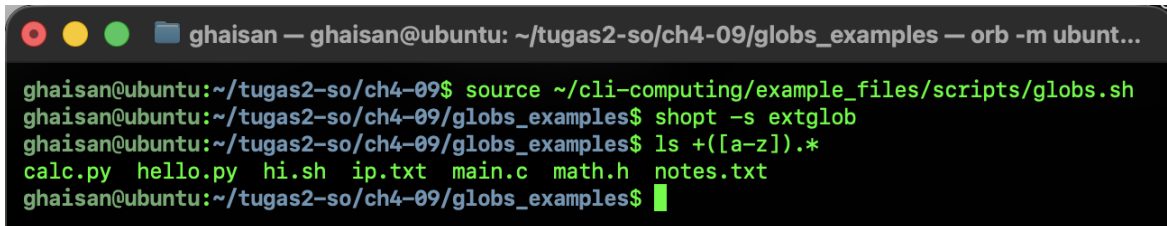
2.9 Soal 9: Nama hanya huruf kecil sebelum ekstensi

Soal. Tampilkan nama berkas yang sebelum ekstensinya hanya berisi huruf kecil.

Jawaban.

```
shopt -s extglob
ls +([a-z]).*
```

+ (pola) berarti pola tersebut muncul satu kali atau lebih, jadi + ([a-z]) adalah rentetan huruf kecil. Sesudah itu .* mengizinkan ekstensi apa pun.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-09/globs_examples — orb -m ubuntu...

ghaisan@ubuntu:~/tugas2-so/ch4-09$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-09/globs_examples$ shopt -s extglob
ghaisan@ubuntu:~/tugas2-so/ch4-09/globs_examples$ ls +([a-z]).*
calc.py hello.py hi.sh ip.txt main.c math.h notes.txt
ghaisan@ubuntu:~/tugas2-so/ch4-09/globs_examples$
```

Gambar 2.9: hasil percobaan soal 9

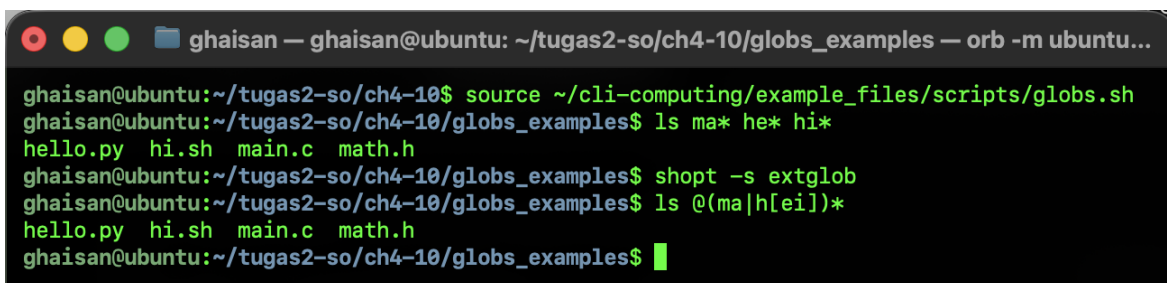
2.10 Soal 10: Nama diawali ma, he, atau hi

Soal. Tampilkan nama berkas yang diawali ma, he, atau hi.

Jawaban.

```
ls ma* he* hi*
shopt -s extglob
ls @(ma|h[ei]).*
```

Beberapa pola boleh ditulis sebagai argumen terpisah karena shell melebarkan semuanya sebelum perintah jalan. Versi extglob @(ma|h[ei]).* melakukan hal yang sama dengan satu pola berisi pilihan.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-10/globs_examples — orb -m ubuntu...

ghaisan@ubuntu:~/tugas2-so/ch4-10$ source ~/cli-computing/example_files/scripts/globs.sh
ghaisan@ubuntu:~/tugas2-so/ch4-10/globs_examples$ ls ma* he* hi*
hello.py hi.sh main.c math.h
ghaisan@ubuntu:~/tugas2-so/ch4-10/globs_examples$ shopt -s extglob
ghaisan@ubuntu:~/tugas2-so/ch4-10/globs_examples$ ls @(ma|h[ei]).*
hello.py hi.sh main.c math.h
ghaisan@ubuntu:~/tugas2-so/ch4-10/globs_examples$
```

Gambar 2.10: hasil percobaan soal 10

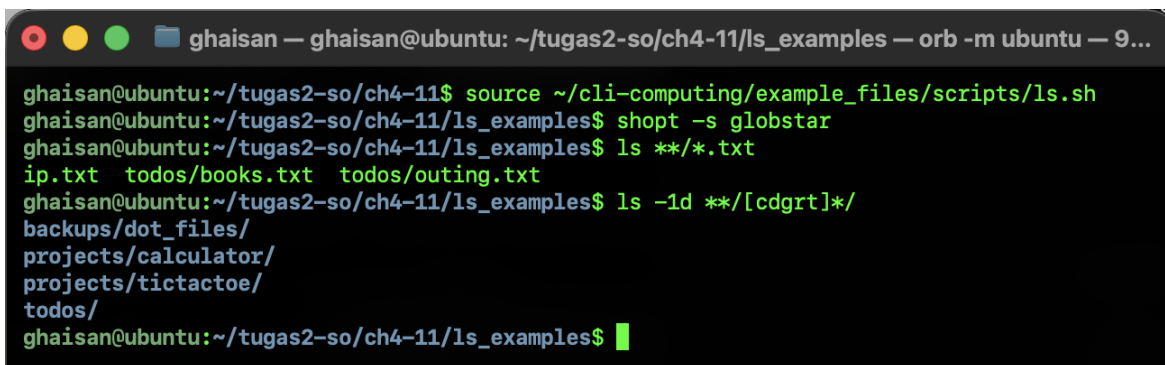
2.11 Soal 11: Menelusuri sub-direktori sampai ke dalam

Soal. Tampilkan berkas berakhiran `.txt` dan direktori yang diawali huruf `c`, `d`, `g`, `r`, atau `t`, tanpa tahu kedalaman sub-direktornya.

Jawaban.

```
source ~/cli-computing/example_files/scripts/ls.sh
shopt -s globstar
ls **/*.txt
ls -ld **/[cdgrt]*/
```

Dengan `globstar` aktif, `**` menelusuri sub-direktori sampai berapa pun dalamnya. Untuk daftar direktori, polanya diakhiri `/` dan dipakai `-d` supaya yang tampil entri direktornya, bukan isinya, sedangkan `-l` membuat satu entri satu baris.

A screenshot of a terminal window with a dark background. The window title is "ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-11/ls_examples — orb -m ubuntu — 9...". The terminal shows the following commands and their output:
ghaisan@ubuntu:~/tugas2-so/ch4-11\$ source ~/cli-computing/example_files/scripts/ls.sh
ghaisan@ubuntu:~/tugas2-so/ch4-11/ls_examples\$ shopt -s globstar
ghaisan@ubuntu:~/tugas2-so/ch4-11/ls_examples\$ ls **/*.txt
ip.txt todos/books.txt todos/outing.txt
ghaisan@ubuntu:~/tugas2-so/ch4-11/ls_examples\$ ls -ld **/[cdgrt]*/
backups/dot_files/
projects/calculator/
projects/tictactoe/
todos/
ghaisan@ubuntu:~/tugas2-so/ch4-11/ls_examples\$

Gambar 2.11: hasil percobaan soal 11

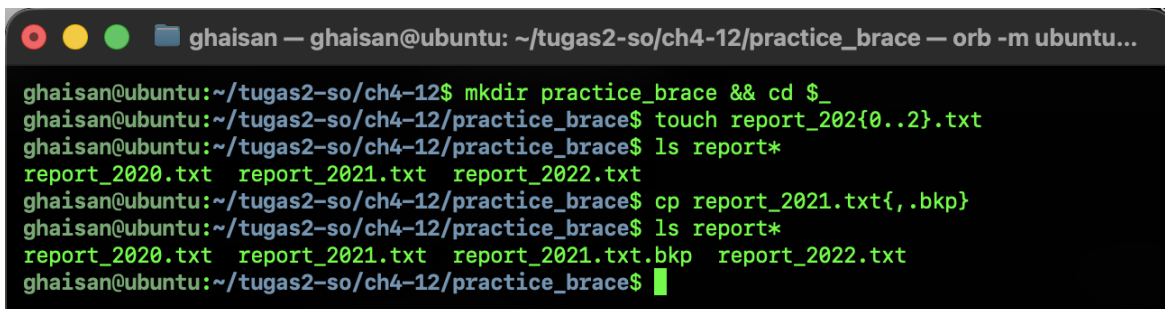
2.12 Soal 12: Brace expansion untuk membuat dan menyalin berkas

Soal. Buat berkas `report_2020.txt` sampai `report_2022.txt`, lalu salin `report_2021.txt` menjadi `report_2021.txt.bkp` memakai brace expansion.

Jawaban.

```
mkdir practice_brace && cd $_
touch report_202{0..2}.txt
ls report*
cp report_2021.txt{,.bkp}
ls report*
```

`{0..2}` melebar menjadi 0, 1, dan 2 sehingga satu perintah `touch` membuat tiga berkas. Pada `cp`, bentuk `report_2021.txt{,.bkp}` melebar jadi dua argumen, yaitu nama asli dan nama berakhiran `.bkp`, sehingga cocok untuk membuat salinan cadangan.



```
ghaisan@ubuntu: ~/tugas2-so/ch4-12/practice_brace — orb -m ubuntu...
ghaisan@ubuntu:~/tugas2-so/ch4-12$ mkdir practice_brace && cd $_
ghaisan@ubuntu:~/tugas2-so/ch4-12/practice_brace$ touch report_202{0..2}.txt
ghaisan@ubuntu:~/tugas2-so/ch4-12/practice_brace$ ls report*
report_2020.txt report_2021.txt report_2022.txt
ghaisan@ubuntu:~/tugas2-so/ch4-12/practice_brace$ cp report_2021.txt{,.bkp}
ghaisan@ubuntu:~/tugas2-so/ch4-12/practice_brace$ ls report*
report_2020.txt report_2021.txt report_2021.txt.bkp report_2022.txt
ghaisan@ubuntu:~/tugas2-so/ch4-12/practice_brace$
```

Gambar 2.12: hasil percobaan soal 12

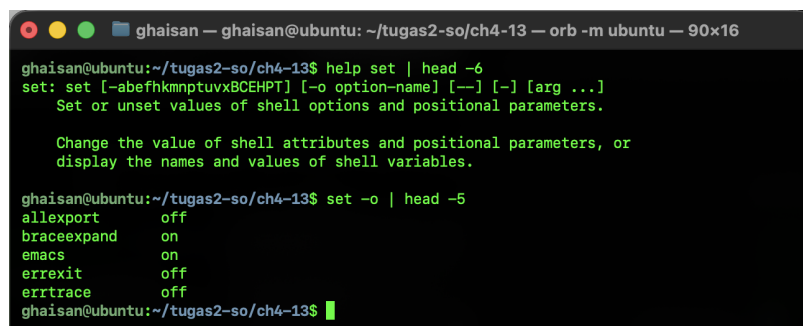
2.13 Soal 13: Perintah bawaan set

Soal. Apa fungsi perintah bawaan `set`?

Jawaban.

```
help set | head -6
set -o | head -5
```

Menurut `help set`, perintah ini dipakai untuk mengubah nilai atribut shell dan parameter posisi, atau menampilkan nama dan nilai variabel shell. Bentuk `set -o` menampilkan daftar opsi shell beserta status aktif atau tidaknya.



```
ghaisan@ubuntu: ~/tugas2-so/ch4-13 — orb -m ubuntu — 90x16
ghaisan@ubuntu:~/tugas2-so/ch4-13$ help set | head -6
set: set [-abefhkmnptuvxBCEHPT] [-o option-name] [--] [-] [arg ...]
    Set or unset values of shell options and positional parameters.

    Change the value of shell attributes and positional parameters, or
    display the names and values of shell variables.

ghaisan@ubuntu:~/tugas2-so/ch4-13$ set -o | head -5
allexport      off
braceexpand    on
emacs          on
errexit        off
errtrace       off
ghaisan@ubuntu:~/tugas2-so/ch4-13$
```

Gambar 2.13: hasil percobaan soal 13

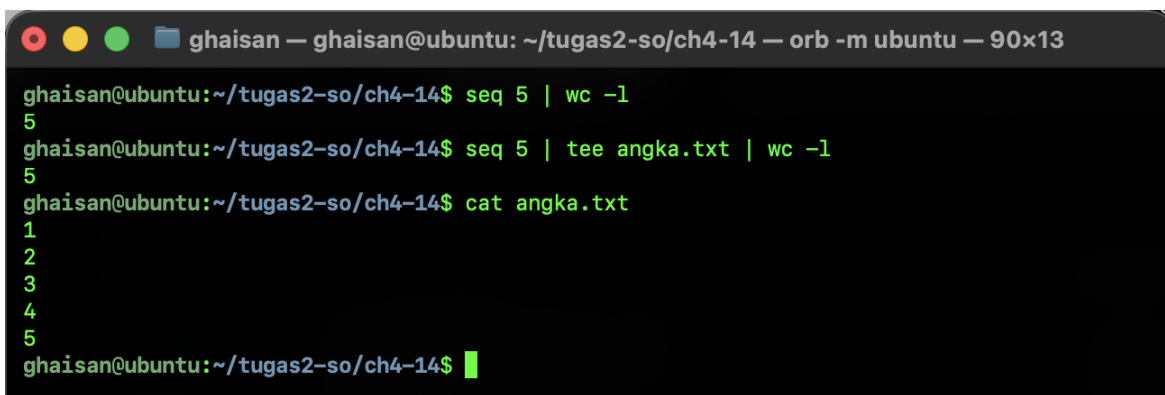
2.14 Soal 14: Operator pipe dan perintah tee

Soal. Apa fungsi operator `|`, dan kapan perlu menambahkan perintah `tee`?

Jawaban.

```
seq 5 | wc -l
seq 5 | tee angka.txt | wc -l
cat angka.txt
```

`|` mengirim keluaran perintah di sebelah kiri menjadi masukan perintah di sebelah kanan. `tee` dipakai kalau keluaran itu ingin disimpan ke berkas sekaligus diteruskan ke perintah berikutnya, seperti contoh di atas yang menyimpan angka ke `angka.txt` tetapi tetap menghitung jumlah barisnya.



```
ghaisan@ubuntu:~/tugas2-so/ch4-14$ seq 5 | wc -l
5
ghaisan@ubuntu:~/tugas2-so/ch4-14$ seq 5 | tee angka.txt | wc -l
5
ghaisan@ubuntu:~/tugas2-so/ch4-14$ cat angka.txt
1
2
3
4
5
ghaisan@ubuntu:~/tugas2-so/ch4-14$
```

Gambar 2.14: hasil percobaan soal 14

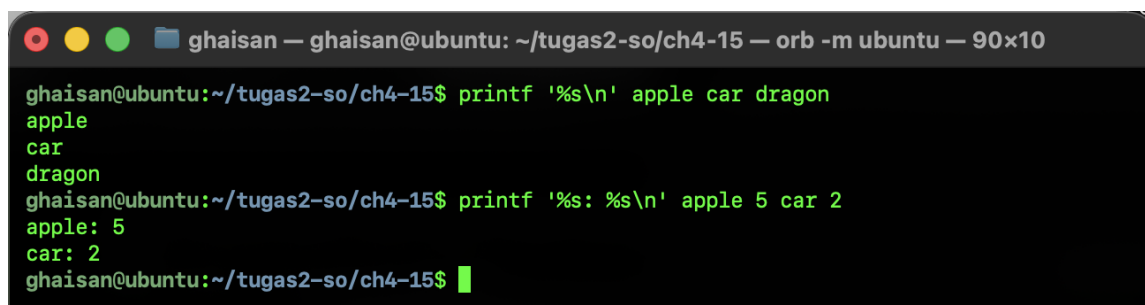
2.15 Soal 15: printf memakai ulang format

Soal. Apa yang dilakukan perintah `printf '%s\n' apple car dragon`?

Jawaban.

```
printf '%s\n' apple car dragon
printf '%s: %s\n' apple 5 car 2
```

Format pada `printf` dipakai ulang sampai semua argumen habis, jadi format `%s\n` membuat tiga argumen tercetak di tiga baris. Contoh kedua memakai dua `%s` sehingga argumen diambil dua-dua.



```
ghaisan@ubuntu:~/tugas2-so/ch4-15$ printf '%s\n' apple car dragon
apple
car
dragon
ghaisan@ubuntu:~/tugas2-so/ch4-15$ printf '%s: %s\n' apple 5 car 2
apple: 5
car: 2
ghaisan@ubuntu:~/tugas2-so/ch4-15$
```

Gambar 2.15: hasil percobaan soal 15

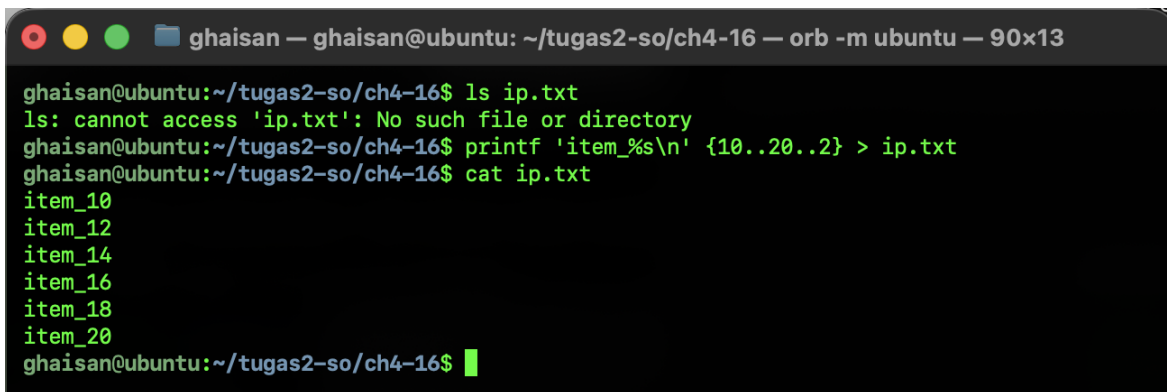
2.16 Soal 16: Membuat berkas dengan brace expansion

Soal. Buat berkas `ip.txt` berisi `item_10` sampai `item_20` dengan langkah 2, memakai brace expansion.

Jawaban.

```
ls ip.txt
printf 'item_%s\n' {10..20..2} > ip.txt
cat ip.txt
```

`{10..20..2}` adalah rentang angka dengan langkah 2. Karena format `printf` dipakai ulang, tiap angka menjadi satu baris `item_NN`, lalu `>` menulis hasilnya ke berkas baru.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-16 — orb -m ubuntu — 90x13
ghaisan@ubuntu:~/tugas2-so/ch4-16$ ls ip.txt
ls: cannot access 'ip.txt': No such file or directory
ghaisan@ubuntu:~/tugas2-so/ch4-16$ printf 'item_%s\n' {10..20..2} > ip.txt
ghaisan@ubuntu:~/tugas2-so/ch4-16$ cat ip.txt
item_10
item_12
item_14
item_16
item_18
item_20
ghaisan@ubuntu:~/tugas2-so/ch4-16$
```

Gambar 2.16: hasil percobaan soal 16

2.17 Soal 17: Menambahkan baris kombinasi ke berkas

Soal. Tambahkan baris `apple_1_banana_6` sampai `apple_3_banana_8` ke `ip.txt` memakai brace expansion.

Jawaban.

```
printf 'item_%s\n' {10..20..2} > ip.txt
printf '%s\n' apple_{1..3}_banana_{6..8} >> ip.txt
cat ip.txt
```

Dua brace dalam satu kata menghasilkan semua kombinasi, jadi $3 \times 3 = 9$ baris. Operator `>>` menambah di akhir berkas, berbeda dengan `>` yang menimpa isinya.



```
ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-17 — orb -m ubuntu — 90x21
ghaisan@ubuntu:~/tugas2-so/ch4-17$ printf 'item_%s\n' {10..20..2} > ip.txt
ghaisan@ubuntu:~/tugas2-so/ch4-17$ printf '%s\n' apple_{1..3}_banana_{6..8} >> ip.txt
ghaisan@ubuntu:~/tugas2-so/ch4-17$ cat ip.txt
item_10
item_12
item_14
item_16
item_18
item_20
apple_1_banana_6
apple_1_banana_7
apple_1_banana_8
apple_2_banana_6
apple_2_banana_7
apple_2_banana_8
apple_3_banana_6
apple_3_banana_7
apple_3_banana_8
ghaisan@ubuntu:~/tugas2-so/ch4-17$
```

Gambar 2.17: hasil percobaan soal 17

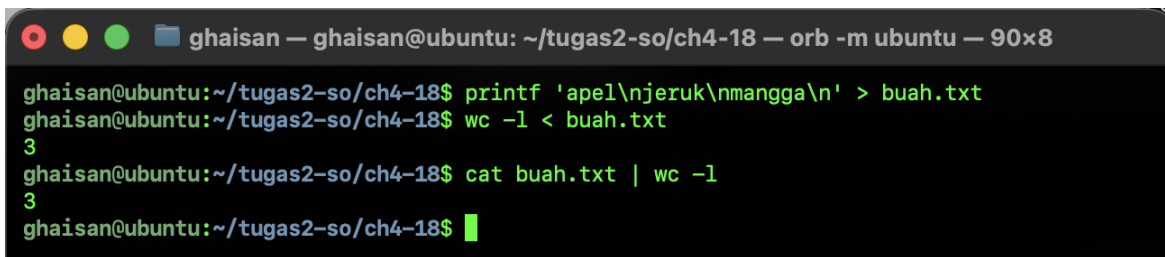
2.18 Soal 18: Perbedaan operator < dan |

Soal. Apa perbedaan operator < dan |?

Jawaban.

```
printf 'apel\njeruk\nmangga\n' > buah.txt
wc -l < buah.txt
cat buah.txt | wc -l
```

< mengambil masukan dari sebuah berkas, sedangkan | mengambil masukan dari keluaran perintah lain. Hasil kedua perintah di atas sama, tetapi yang memakai pipe menjalankan satu proses tambahan yaitu `cat`.

A screenshot of a terminal window with a dark background. The window title is 'ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-18 — orb -m ubuntu — 90x8'. The terminal shows the following commands and their outputs: 1. 'ghaisan@ubuntu:~/tugas2-so/ch4-18\$ printf 'apel\njeruk\nmangga\n' > buah.txt' 2. 'ghaisan@ubuntu:~/tugas2-so/ch4-18\$ wc -l < buah.txt' followed by the output '3'. 3. 'ghaisan@ubuntu:~/tugas2-so/ch4-18\$ cat buah.txt | wc -l' followed by the output '3'. 4. 'ghaisan@ubuntu:~/tugas2-so/ch4-18\$' followed by a green cursor.

```
ghaisan@ubuntu:~/tugas2-so/ch4-18$ printf 'apel\njeruk\nmangga\n' > buah.txt
ghaisan@ubuntu:~/tugas2-so/ch4-18$ wc -l < buah.txt
3
ghaisan@ubuntu:~/tugas2-so/ch4-18$ cat buah.txt | wc -l
3
ghaisan@ubuntu:~/tugas2-so/ch4-18$
```

Gambar 2.18: hasil percobaan soal 18

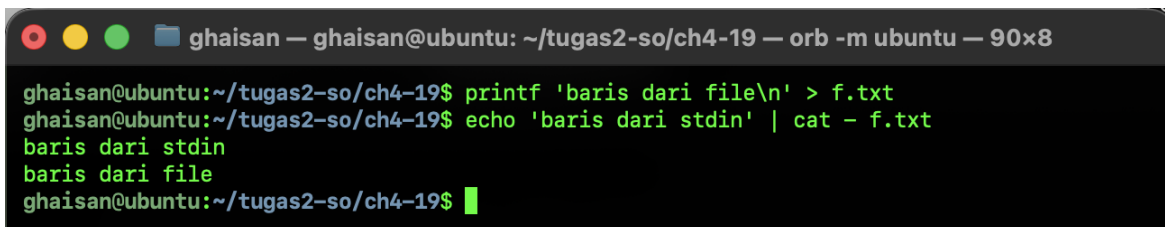
2.19 Soal 19: Karakter untuk mewakili stdin

Soal. Karakter apa yang biasa dipakai untuk mewakili data `stdin` pada posisi argumen berkas?

Jawaban.

```
printf 'baris dari file\n' > f.txt
echo 'baris dari stdin' | cat - f.txt
```

Karakter `-` mewakili `stdin` di posisi argumen berkas. Pada contoh di atas `cat` membaca `stdin` lebih dulu karena `-` ditulis sebelum nama berkas.

A screenshot of a terminal window with a dark background. The window title is 'ghaisan — ghaisan@ubuntu: ~/tugas2-so/ch4-19 — orb -m ubuntu — 90x8'. The terminal shows the following commands and their outputs: 1. 'ghaisan@ubuntu:~/tugas2-so/ch4-19\$ printf 'baris dari file\n' > f.txt' 2. 'ghaisan@ubuntu:~/tugas2-so/ch4-19\$ echo 'baris dari stdin' | cat - f.txt' followed by the output 'baris dari stdin' on the next line, then 'baris dari file' on the following line. 3. 'ghaisan@ubuntu:~/tugas2-so/ch4-19\$' followed by a green cursor.

```
ghaisan@ubuntu:~/tugas2-so/ch4-19$ printf 'baris dari file\n' > f.txt
ghaisan@ubuntu:~/tugas2-so/ch4-19$ echo 'baris dari stdin' | cat - f.txt
baris dari stdin
baris dari file
ghaisan@ubuntu:~/tugas2-so/ch4-19$
```

Gambar 2.19: hasil percobaan soal 19

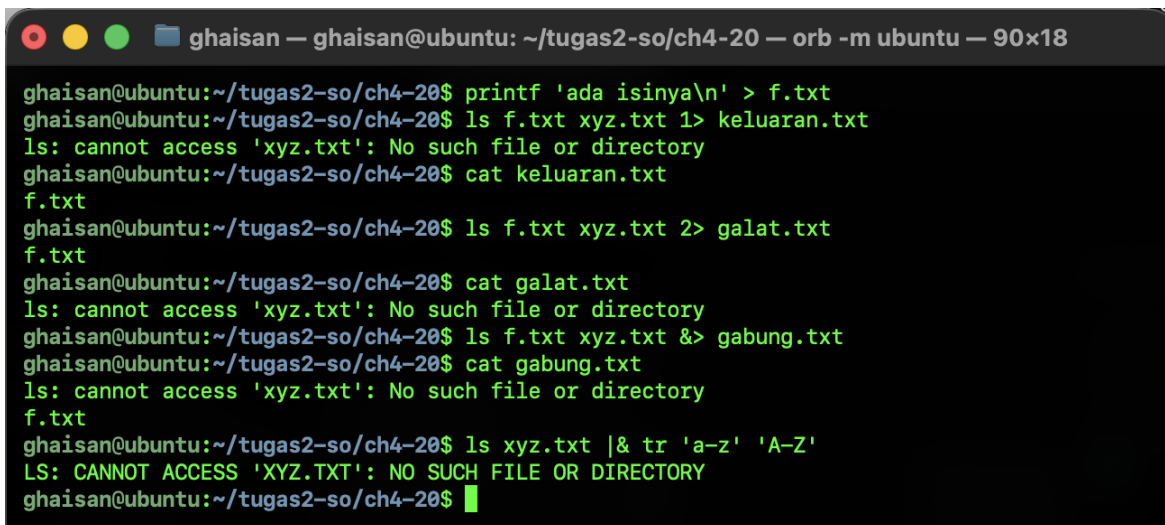
2.20 Soal 20: Operator pengalihan keluaran

Soal. Apa fungsi operator `1>`, `2>`, `&>`, `&>>`, dan `|&`?

Jawaban.

```
printf 'ada isinya\n' > f.txt
ls f.txt xyz.txt 1> keluaran.txt
cat keluaran.txt
ls f.txt xyz.txt 2> galat.txt
cat galat.txt
ls f.txt xyz.txt &> gabung.txt
cat gabung.txt
ls xyz.txt |& tr 'a-z' 'A-Z'
```

`1>` mengalihkan stdout ke berkas, `2>` mengalihkan stderr, `&>` mengalihkan keduanya dan menimpa berkas, `&>>` mengalihkan keduanya tetapi menambah di akhir berkas, dan `|&` mengirim stdout sekaligus stderr ke perintah berikutnya. Terlihat di layar, pesan galat tetap muncul saat hanya stdout yang dialihkan, dan sebaliknya.



```
ghaisan@ubuntu:~/tugas2-so/ch4-20$ printf 'ada isinya\n' > f.txt
ghaisan@ubuntu:~/tugas2-so/ch4-20$ ls f.txt xyz.txt 1> keluaran.txt
ls: cannot access 'xyz.txt': No such file or directory
ghaisan@ubuntu:~/tugas2-so/ch4-20$ cat keluaran.txt
f.txt
ghaisan@ubuntu:~/tugas2-so/ch4-20$ ls f.txt xyz.txt 2> galat.txt
f.txt
ghaisan@ubuntu:~/tugas2-so/ch4-20$ cat galat.txt
ls: cannot access 'xyz.txt': No such file or directory
ghaisan@ubuntu:~/tugas2-so/ch4-20$ ls f.txt xyz.txt &> gabung.txt
ghaisan@ubuntu:~/tugas2-so/ch4-20$ cat gabung.txt
ls: cannot access 'xyz.txt': No such file or directory
f.txt
ghaisan@ubuntu:~/tugas2-so/ch4-20$ ls xyz.txt |& tr 'a-z' 'A-Z'
LS: CANNOT ACCESS 'XYZ.TXT': NO SUCH FILE OR DIRECTORY
ghaisan@ubuntu:~/tugas2-so/ch4-20$
```

Gambar 2.20: hasil percobaan soal 20

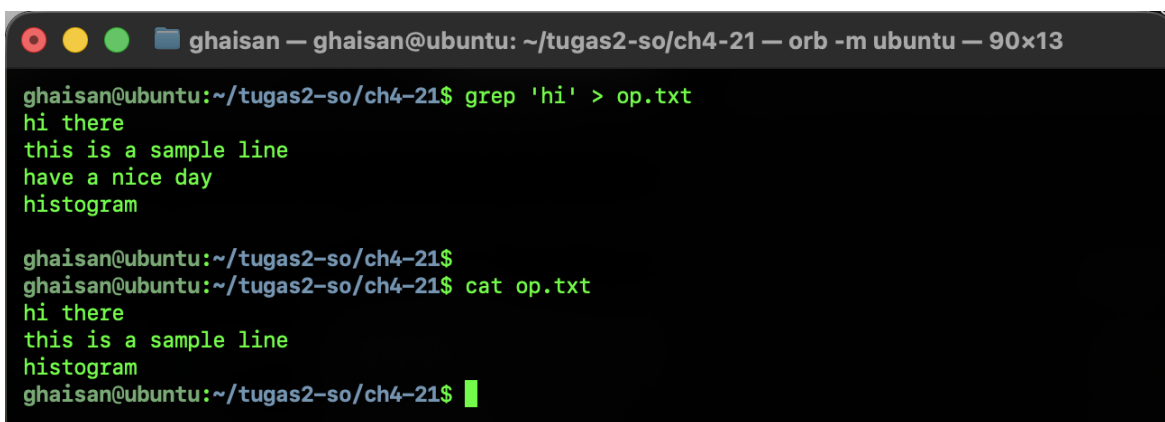
2.21 Soal 21: Masukan dari keyboard sampai Ctrl+D

Soal. Apa isi `op.txt` kalau perintah `grep 'hi' > op.txt` diketik lalu diisi empat baris dan diakhiri Ctrl+D?

Jawaban.

```
grep 'hi' > op.txt
(ketik empat baris, lalu tekan Ctrl+D)
cat op.txt
```

Karena tidak diberi nama berkas, `grep` membaca dari keyboard sampai Ctrl+D menandakan akhir masukan. Yang tersimpan hanya baris yang memuat `hi`, termasuk baris `this is a sample line` karena ada `hi` di dalam kata `this`.



```
ghaisan@ubuntu: ~/tugas2-so/ch4-21 — orb -m ubuntu — 90x13
ghaisan@ubuntu:~/tugas2-so/ch4-21$ grep 'hi' > op.txt
hi there
this is a sample line
have a nice day
histogram

ghaisan@ubuntu:~/tugas2-so/ch4-21$
ghaisan@ubuntu:~/tugas2-so/ch4-21$ cat op.txt
hi there
this is a sample line
histogram
ghaisan@ubuntu:~/tugas2-so/ch4-21$
```

Gambar 2.21: hasil percobaan soal 21

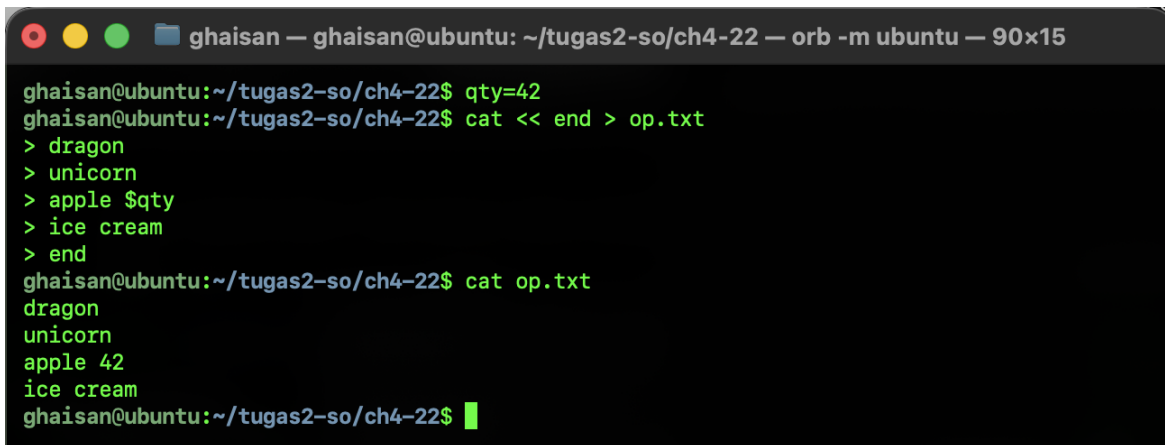
2.22 Soal 22: Here document

Soal. Apa isi `op.txt` setelah perintah `cat << end > op.txt` dengan variabel `qty=42`?

Jawaban.

```
qty=42
cat << end > op.txt
(isi empat baris, ditutup dengan baris end)
cat op.txt
```

`<< end` membaca baris demi baris sampai menemukan penanda `end`. Karena penandanya tidak dikutip, `$qty` tetap diganti nilainya menjadi 42. Kalau isinya ingin apa adanya, penandanya ditulis `<< 'end'` atau `<< \end`.



```
ghaisan@ubuntu:~/tugas2-so/ch4-22$ qty=42
ghaisan@ubuntu:~/tugas2-so/ch4-22$ cat << end > op.txt
> dragon
> unicorn
> apple $qty
> ice cream
> end
ghaisan@ubuntu:~/tugas2-so/ch4-22$ cat op.txt
dragon
unicorn
apple 42
ice cream
ghaisan@ubuntu:~/tugas2-so/ch4-22$
```

Gambar 2.22: hasil percobaan soal 22

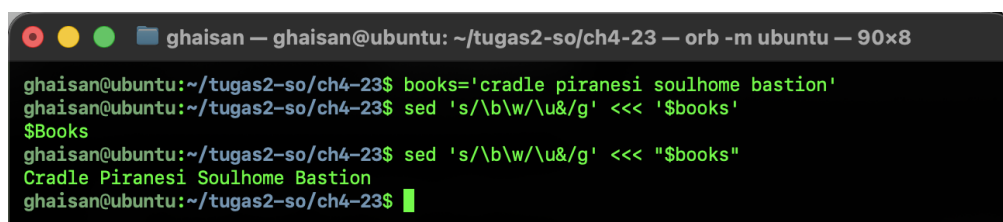
2.23 Soal 23: Here string dan interpolasi variabel

Soal. Perbaiki perintah `sed 's/\b\w/\u&/g' <<< '$books'` supaya hasilnya `Cradle Piranesi Soulhome Bastion`.

Jawaban.

```
books='cradle piranesi soulhome bastion'
sed 's/\b\w/\u&/g' <<< "$books"
```

`<<<` mengirim satu string sebagai stdin. Petik tunggal membuat `$books` dianggap teks biasa sehingga yang diproses justru tulisan `$books`. Dengan petik ganda, isi variabelnya yang dikirim ke `sed`.



```
ghaisan@ubuntu:~/tugas2-so/ch4-23$ books='cradle piranesi soulhome bastion'
ghaisan@ubuntu:~/tugas2-so/ch4-23$ sed 's/\b\w/\u&/g' <<< '$books'
$Books
ghaisan@ubuntu:~/tugas2-so/ch4-23$ sed 's/\b\w/\u&/g' <<< "$books"
Cradle Piranesi Soulhome Bastion
ghaisan@ubuntu:~/tugas2-so/ch4-23$
```

Gambar 2.23: hasil percobaan soal 23

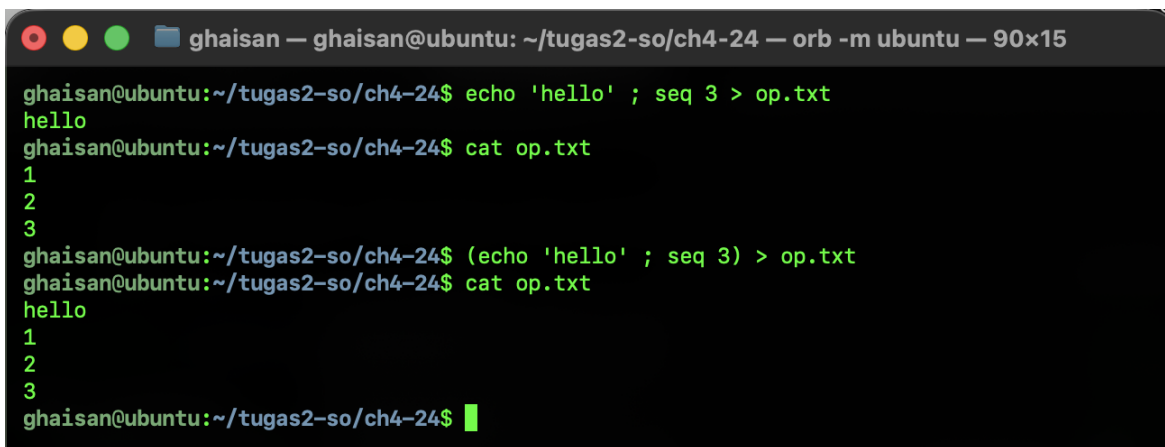
2.24 Soal 24: Mengalihkan keluaran beberapa perintah sekaligus

Soal. Perbaiki perintah `echo 'hello' ; seq 3 > op.txt` supaya `op.txt` memuat `hello` dan angka 1 sampai 3.

Jawaban.

```
echo 'hello' ; seq 3 > op.txt
cat op.txt
(echo 'hello' ; seq 3) > op.txt
cat op.txt
```

Pada perintah awal, tanda `>` hanya menempel pada `seq 3` sehingga `hello` tetap tercetak di layar. Supaya keluaran kedua perintah masuk ke berkas yang sama, keduanya dikelompokkan dulu dengan `( ... )` atau `{ ...; }` baru dialihkan.



```
ghaisan@ubuntu:~/tugas2-so/ch4-24$ echo 'hello' ; seq 3 > op.txt
hello
ghaisan@ubuntu:~/tugas2-so/ch4-24$ cat op.txt
1
2
3
ghaisan@ubuntu:~/tugas2-so/ch4-24$ (echo 'hello' ; seq 3) > op.txt
ghaisan@ubuntu:~/tugas2-so/ch4-24$ cat op.txt
hello
1
2
3
ghaisan@ubuntu:~/tugas2-so/ch4-24$
```

Gambar 2.24: hasil percobaan soal 24

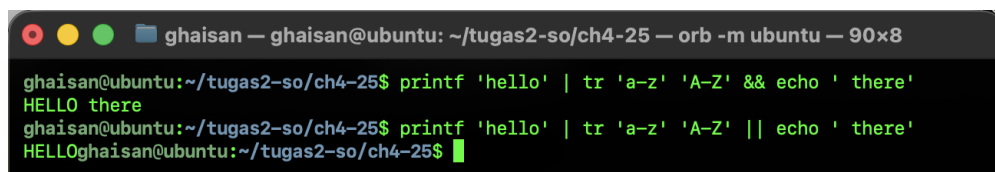
2.25 Soal 25: Operator `&&` dan `||`

Soal. Apa keluaran perintah `printf 'hello' | tr 'a-z' 'A-Z' && echo ' there'` dan versi `||`-nya?

Jawaban.

```
printf 'hello' | tr 'a-z' 'A-Z' && echo ' there'
printf 'hello' | tr 'a-z' 'A-Z' || echo ' there'
```

Keduanya melihat status keluar perintah sebelumnya. Perintah `tr` berhasil dengan status 0, jadi `&&` menjalankan `echo ' there'` sedangkan `||` melewatinya. Karena `printf 'hello'` tidak memberi baris baru, prompt menempel langsung sesudah tulisan HELLO.



```
ghaisan@ubuntu:~/tugas2-so/ch4-25$ printf 'hello' | tr 'a-z' 'A-Z' && echo ' there'
HELLO there
ghaisan@ubuntu:~/tugas2-so/ch4-25$ printf 'hello' | tr 'a-z' 'A-Z' || echo ' there'
HELLOghaisan@ubuntu:~/tugas2-so/ch4-25$
```

Gambar 2.25: hasil percobaan soal 25

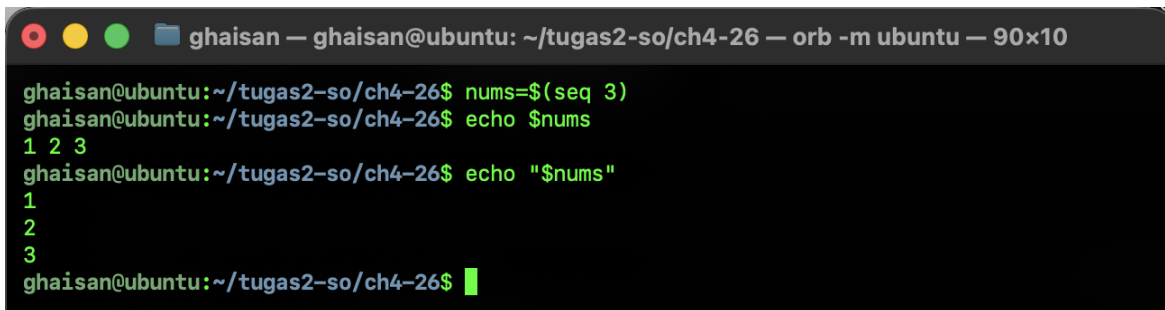
2.26 Soal 26: Menjaga baris baru hasil command substitution

Soal. Perbaiki perintah supaya isi variabel `nums=$(seq 3)` tercetak tiga baris.

Jawaban.

```
nums=$(seq 3)
echo $nums
echo "$nums"
```

Tanpa petik, hasil `$nums` dipecah oleh shell berdasarkan spasi dan baris baru lalu digabung dengan satu spasi, sehingga tercetak satu baris. Dengan petik ganda, isinya diteruskan apa adanya sehingga tetap tiga baris.



```
ghaisan@ubuntu:~/tugas2-so/ch4-26$ nums=$(seq 3)
ghaisan@ubuntu:~/tugas2-so/ch4-26$ echo $nums
1 2 3
ghaisan@ubuntu:~/tugas2-so/ch4-26$ echo "$nums"
1
2
3
ghaisan@ubuntu:~/tugas2-so/ch4-26$
```

Gambar 2.26: hasil percobaan soal 26

2.27 Soal 27: Process substitution dan brace expansion

Soal. Apakah `paste -d, <(seq 3) <(printf '%s\n' item_{1..3})` dan `printf '%s\n' {1..3},item_{1..3}` menghasilkan keluaran yang sama?

Jawaban.

```
paste -d, <(seq 3) <(printf '%s\n' item_{1..3})
printf '%s\n' {1..3},item_{1..3}
```

Tidak sama. `paste` menggabungkan baris ke baris dari dua masukan sehingga hasilnya 3 baris. Sedangkan dua brace dalam satu kata menghasilkan semua kombinasi, jadi keluarannya 9 baris.



```
ghaisan@ubuntu:~/tugas2-so/ch4-27$ paste -d, <(seq 3) <(printf '%s\n' item_{1..3})
1,item_1
2,item_2
3,item_3
ghaisan@ubuntu:~/tugas2-so/ch4-27$ printf '%s\n' {1..3},item_{1..3}
1,item_1
1,item_2
1,item_3
2,item_1
2,item_2
2,item_3
3,item_1
3,item_2
3,item_3
ghaisan@ubuntu:~/tugas2-so/ch4-27$
```

Gambar 2.27: hasil percobaan soal 27

BAB III CHAPTER 5: VIEWING PART OR WHOLE FILE CONTENTS

Bagian ini berisi 12 soal chapter 5 yang seluruhnya dikerjakan di folder `example_files/text_files` bawaan buku.

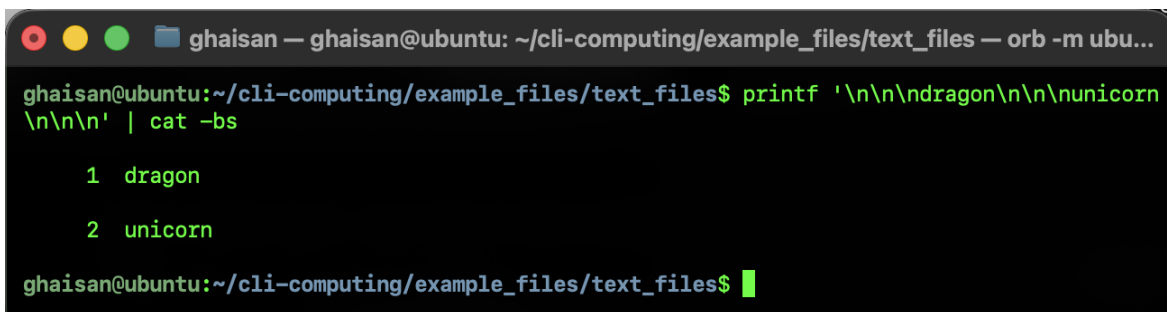
3.1 Soal 1: Memberi nomor baris tanpa baris kosong berulang

Soal. Opsi apa yang dipakai supaya `cat` menomori baris berisi teks saja dan memampatkan baris kosong berturut-turut?

Jawaban.

```
printf '\n\nndragon\n\n\nunicorn\n\n\n' | cat -bs
```

`-b` memberi nomor hanya pada baris yang tidak kosong, sedangkan `-s` memampatkan baris kosong berturut-turut menjadi satu baris kosong.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf '\n\nndragon\n\n\nunicorn\n\n\n' | cat -bs

1 dragon

2 unicorn

ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.1: hasil percobaan soal 1

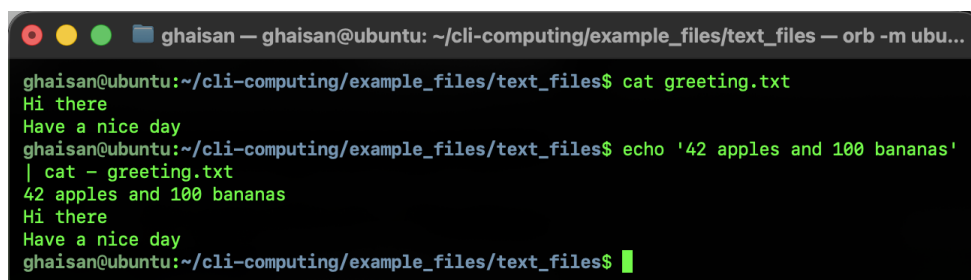
3.2 Soal 2: Menggabungkan stdin dengan isi berkas

Soal. Argumen apa yang diberikan ke `cat` supaya keluaran `echo` tampil lebih dulu, baru isi `greeting.txt`?

Jawaban.

```
cat greeting.txt
echo '42 apples and 100 bananas' | cat - greeting.txt
```

Karakter `-` mewakili stdin di posisi argumen berkas, jadi urutan `- greeting.txt` membuat data dari pipe dicetak dulu, baru isi berkasnya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat greeting.txt
Hi there
Have a nice day
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo '42 apples and 100 bananas'
42 apples and 100 bananas
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.2: hasil percobaan soal 2

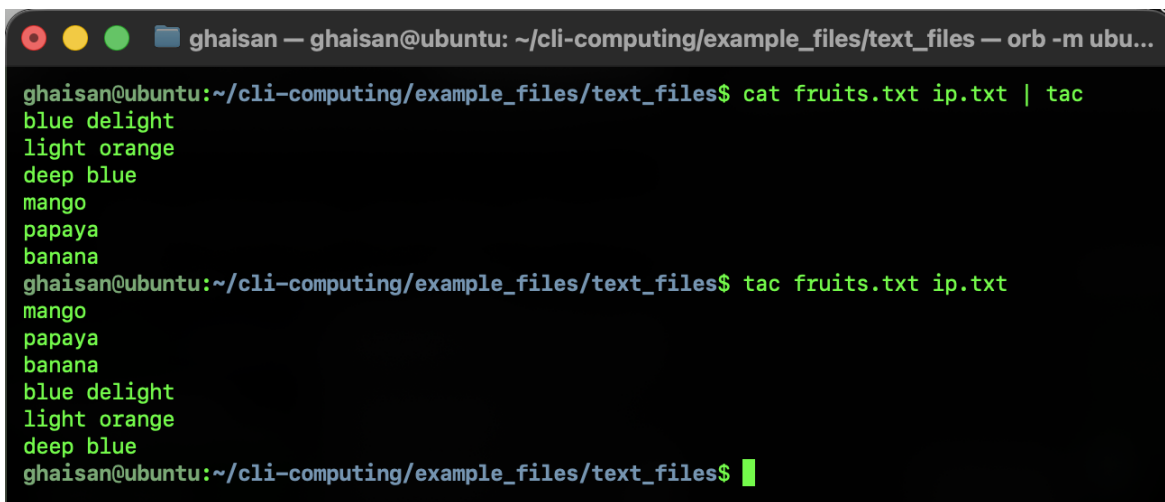
3.3 Soal 3: Membalik urutan baris gabungan atau per berkas

Soal. Apakah `cat fruits.txt ip.txt | tac` dan `tac fruits.txt ip.txt` menghasilkan keluaran yang sama?

Jawaban.

```
cat fruits.txt ip.txt | tac
tac fruits.txt ip.txt
```

Tidak sama. Pada perintah pertama, `tac` menerima satu aliran gabungan sehingga urutan seluruh baris dibalik. Pada perintah kedua, `tac` membalik isi tiap berkas secara terpisah lalu mencetaknya berurutan sesuai urutan argumen.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat fruits.txt ip.txt | tac
blue delight
light orange
deep blue
mango
papaya
banana
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ tac fruits.txt ip.txt
mango
papaya
banana
blue delight
light orange
deep blue
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.3: hasil percobaan soal 3

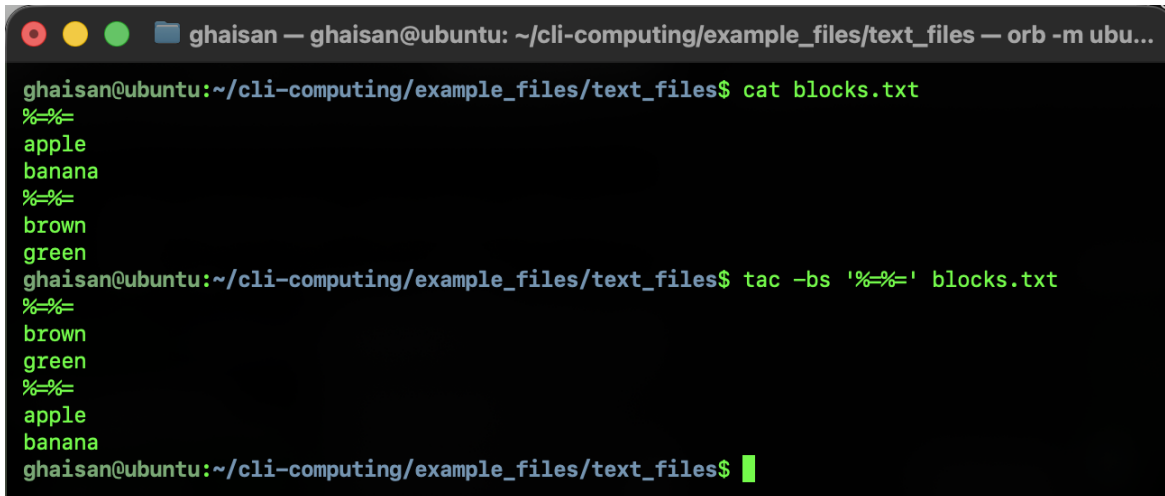
3.4 Soal 4: Membalik blok teks dengan pemisah khusus

Soal. Balik urutan blok pada `blocks.txt` yang dipisahkan penanda `%=%=`.

Jawaban.

```
cat blocks.txt
tac -bs '%=%=' blocks.txt
```

`-s` mengganti pemisah dari baris baru menjadi teks `%=%=`, sedangkan `-b` membuat pemisah ditempel di depan blok, bukan di belakangnya, sehingga tampilannya sama seperti berkas aslinya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat blocks.txt
%=%=
apple
banana
%=%=
brown
green
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ tac -bs '%=%=' blocks.txt
%=%=
brown
green
%=%=
apple
banana
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.4: hasil percobaan soal 4

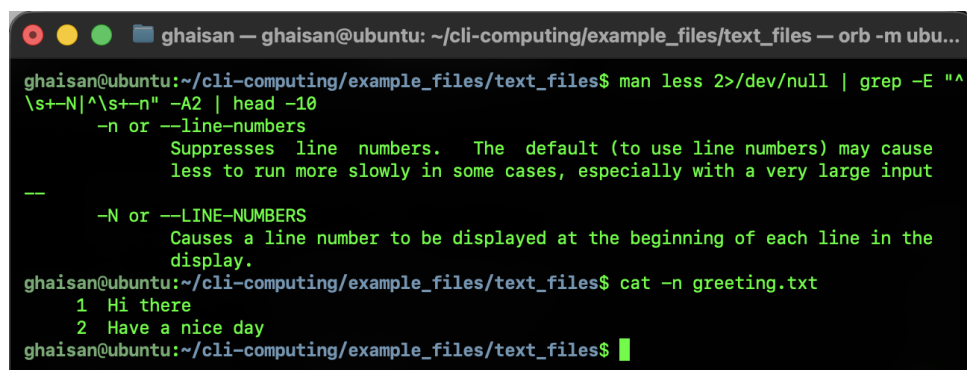
3.5 Soal 5: Opsi `-n` dan `-N` pada `less`

Soal. Apa beda `less -n` dan `less -N`, dan apakah `cat -n` mirip `less -n`?

Jawaban.

```
man less 2>/dev/null | grep -E "\s+-N|\s+-n" -A2 | head -10
cat -n greeting.txt
```

`-N` menyalakan nomor baris, sedangkan `-n` mematikannya. Karena `cat -n` justru menomori baris, fungsinya mirip `less -N` dan berkebalikan dengan `less -n`.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ man less 2>/dev/null | grep -E "\s+-N|\s+-n" -A2 | head -10
-n or --line-numbers
    Suppresses line numbers. The default (to use line numbers) may cause
    less to run more slowly in some cases, especially with a very large input
--
-N or --LINE-NUMBERS
    Causes a line number to be displayed at the beginning of each line in the
    display.
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat -n greeting.txt
1 Hi there
2 Have a nice day
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.5: hasil percobaan soal 5

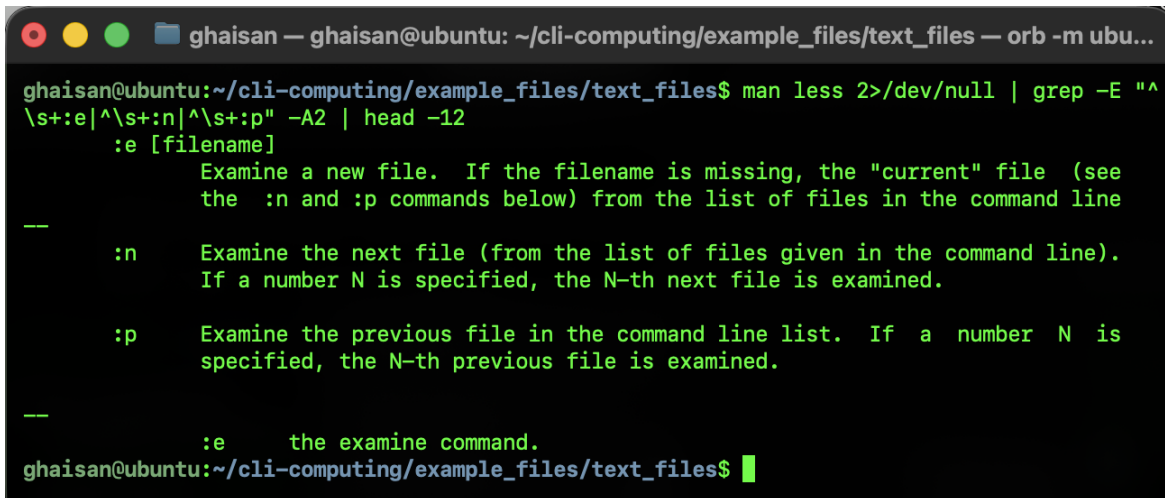
3.6 Soal 6: Membuka berkas lain dari dalam less

Soal. Perintah apa yang dipakai untuk membuka berkas lain dari dalam sesi `less`, dan untuk berpindah ke berkas sebelumnya atau berikutnya?

Jawaban.

```
man less 2>/dev/null | grep -E "\s+:e|\s+:n|\s+:p" -A2 | head -12
```

`:e` nama_berkas membuka berkas lain di sesi yang sedang berjalan. Untuk berpindah antar berkas dipakai `:n` ke berkas berikutnya dan `:p` ke berkas sebelumnya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ man less 2>/dev/null | grep -E "\s+:e|\s+:n|\s+:p" -A2 | head -12
:e [filename]
    Examine a new file.  If the filename is missing, the "current" file (see
    the :n and :p commands below) from the list of files in the command line
--
:n      Examine the next file (from the list of files given in the command line).
    If a number N is specified, the N-th next file is examined.

:p      Examine the previous file in the command line list.  If a number N is
    specified, the N-th previous file is examined.

--
:e      the examine command.
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.6: hasil percobaan soal 6

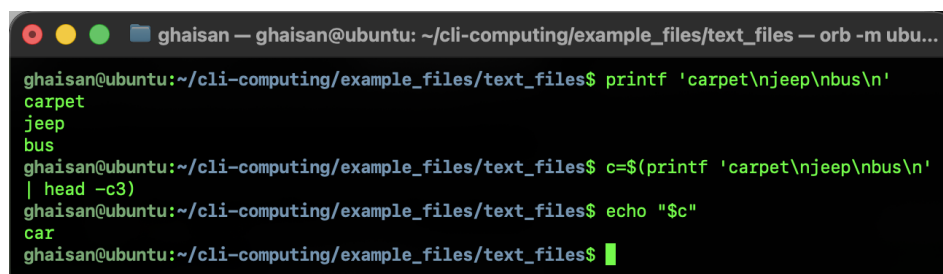
3.7 Soal 7: Mengambil tiga karakter pertama ke variabel

Soal. Simpan tiga karakter pertama dari keluaran `printf 'carpet\njeep\nbus\n'` ke variabel `c`.

Jawaban.

```
printf 'carpet\njeep\nbus\n'
c=$(printf 'carpet\njeep\nbus\n' | head -c3)
echo "$c"
```

`head -c3` mengambil tiga byte pertama, dan `$ ( )` menyimpan hasilnya ke variabel. Karena tiga byte pertama adalah `car`, itulah isi variabel `c`.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'carpet\njeep\nbus\n'
carpet
jeep
bus
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ c=$(printf 'carpet\njeep\nbus\n'
| head -c3)
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo "$c"
car
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.7: hasil percobaan soal 7

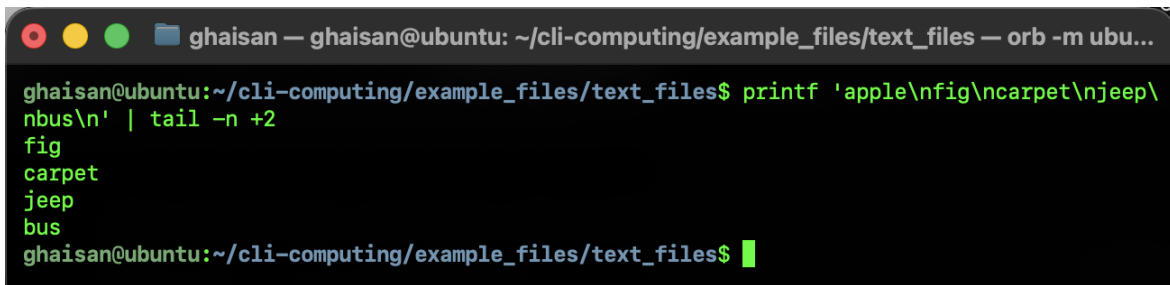
3.8 Soal 8: Menampilkan semua baris kecuali baris pertama

Soal. Bagaimana cara menampilkan semua baris masukan kecuali baris pertama?

Jawaban.

```
printf 'apple\nfig\ncarpet\njeep\nbus\n' | tail -n +2
```

Bentuk `-n +N` pada `tail` berarti mulai mencetak dari baris ke-N, jadi `+2` melewati baris pertama.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'apple\nfig\ncarpet\njeep\nbus\n' | tail -n +2
fig
carpet
jeep
bus
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.8: hasil percobaan soal 8

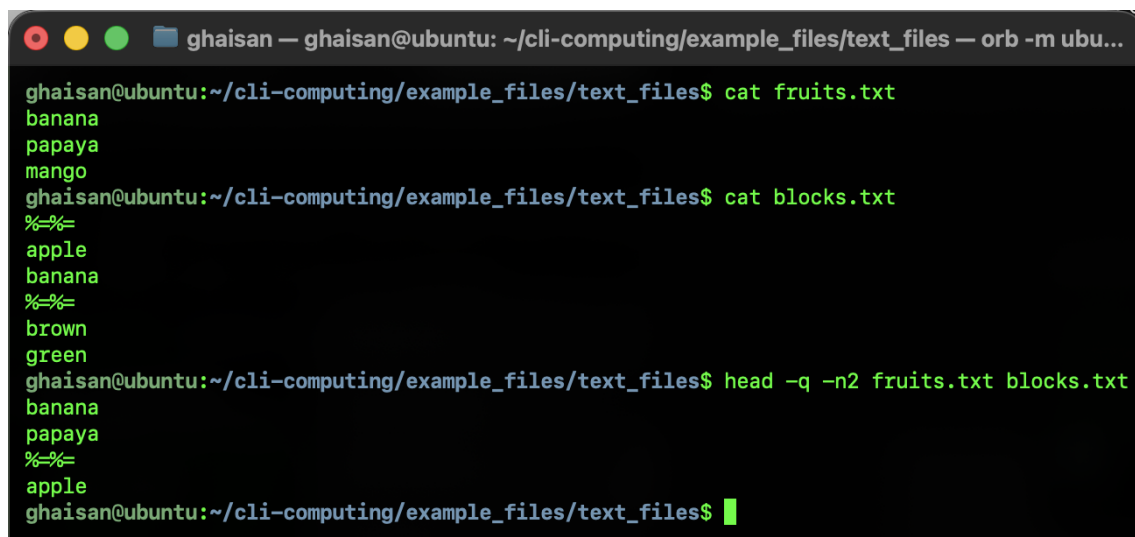
3.9 Soal 9: Dua baris pertama dari beberapa berkas

Soal. Tampilkan dua baris pertama dari `fruits.txt` dan `blocks.txt` tanpa judul nama berkas.

Jawaban.

```
cat fruits.txt
cat blocks.txt
head -q -n2 fruits.txt blocks.txt
```

`-n2` mengambil dua baris pertama tiap berkas. Kalau berkasnya lebih dari satu, `head` biasanya mencetak judul berisi nama berkas, dan `-q` menghilangkan judul tersebut.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat fruits.txt
banana
papaya
mango
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ cat blocks.txt
%=%=
apple
banana
%=%=
brown
green
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ head -q -n2 fruits.txt blocks.txt
banana
papaya
%=%=
apple
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.9: hasil percobaan soal 9

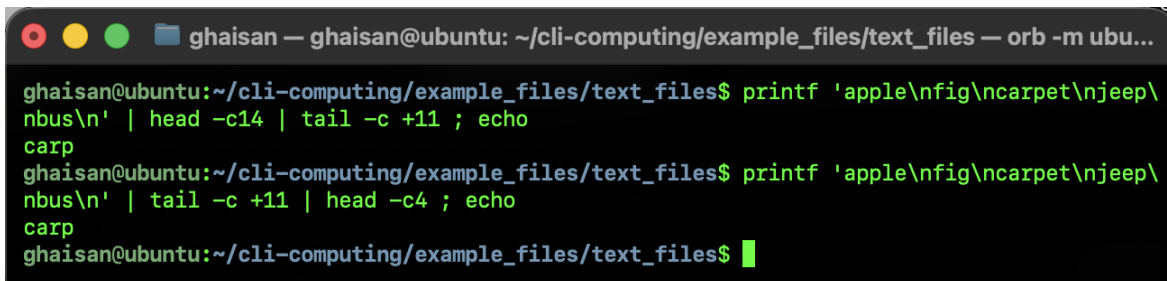
3.10 Soal 10: Mengambil karakter ke-11 sampai ke-14

Soal. Gabungkan `head` dan `tail` untuk mengambil karakter ke-11 sampai ke-14 dari masukan.

Jawaban.

```
printf 'apple\nfig\ncarpet\njeep\nbus\n' | head -c14 | tail -c +11 ; echo  
printf 'apple\nfig\ncarpet\njeep\nbus\n' | tail -c +11 | head -c4 ; echo
```

Perintah pertama memotong sampai byte ke-14 lalu mengambil mulai byte ke-11. Urutan sebaliknya juga bisa, yaitu ambil mulai byte ke-11 lalu batasi 4 byte. `echo` di belakang hanya menambah baris baru supaya prompt tidak menempel.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'apple\nfig\ncarpet\njeep\nbus\n' | head -c14 | tail -c +11 ; echo  
carp  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'apple\nfig\ncarpet\njeep\nbus\n' | tail -c +11 | head -c4 ; echo  
carp  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.10: hasil percobaan soal 10

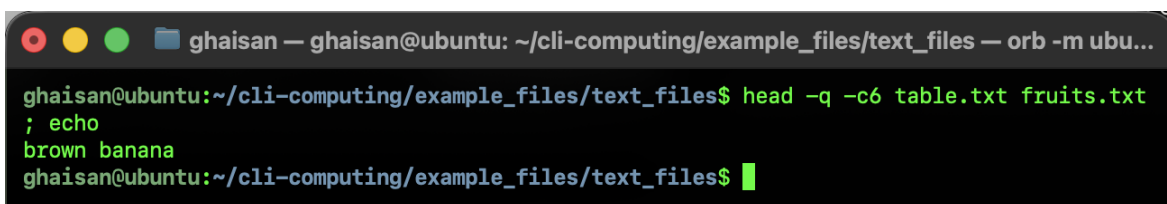
3.11 Soal 11: Enam byte pertama dari dua berkas

Soal. Ambil enam byte pertama dari berkas `table.txt` dan `fruits.txt`.

Jawaban.

```
head -q -c6 table.txt fruits.txt ; echo
```

`-c6` mengambil enam byte pertama tiap berkas dan `-q` menghilangkan judul nama berkas, sehingga hasil dua berkas tercetak menyambung.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ head -q -c6 table.txt fruits.txt ; echo  
brown banana  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.11: hasil percobaan soal 11

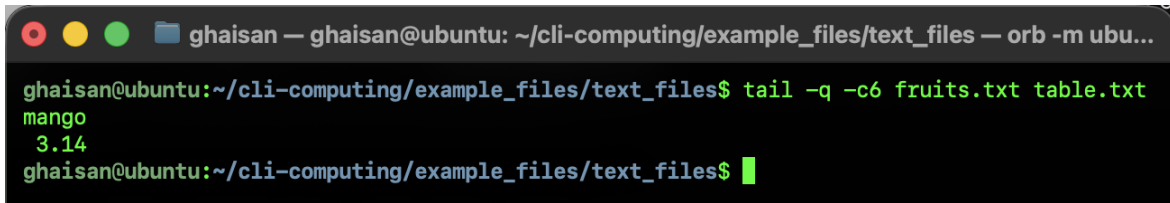
3.12 Soal 12: Enam byte terakhir dari dua berkas

Soal. Ambil enam byte terakhir dari berkas `fruits.txt` dan `table.txt`.

Jawaban.

```
tail -q -c6 fruits.txt table.txt
```

`tail -c6` mengambil enam byte terakhir tiap berkas, dan `-q` sekali lagi menghilangkan judul nama berkas.



```
ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/text_files — orb -m ubu...  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ tail -q -c6 fruits.txt table.txt  
mango  
3.14  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 3.12: hasil percobaan soal 12

BAB IV PENUTUP

4.1 Catatan Hasil

Seluruh keluaran perintah pada 39 soal cocok dengan contoh keluaran di buku. Beberapa hal yang perlu diperhatikan selama pengerjaan:

1. Extended glob (`!(...)`, `+(...)`, `@(...)`) dan globstar (`**`) harus dinyalakan dulu dengan `shopt -s extglob` atau `shopt -s globstar`. Kalau lupa, polanya justru dianggap teks biasa dan perintahnya gagal menemukan berkas.
2. Perintah `ls` menampilkan hasilnya berkolom saat keluar ke terminal, tetapi menjadi satu entri satu baris kalau dialihkan ke pipe atau berkas. Jadi tampilan di tangkapan layar bisa berbeda dengan hasil yang disimpan ke berkas, walaupun isinya sama.
3. `printf` tanpa `\n` membuat prompt menempel di belakang keluaran, seperti terlihat pada soal 25 chapter 4 dan soal 10 chapter 5.
4. Penanda here document yang tidak dikutip tetap mengganti variabel, sedangkan here string dengan petik tunggal tidak. Keduanya sering tertukar dan hasilnya baru kelihatan setelah dicoba.

4.2 Kesimpulan

Chapter 4 memperlihatkan bahwa sebagian besar pekerjaan di baris perintah justru dikerjakan oleh shell sebelum program dijalankan, mulai dari quoting, pelebaran wildcard dan brace, sampai pengalihan masukan dan keluaran. Memahami urutan ini membuat perintah yang salah lebih mudah dilacak, karena kesalahannya sering ada di tahap pelebaran, bukan di programnya. Chapter 5 melengkapinya dengan perintah untuk melihat isi berkas, baik seluruhnya maupun sebagian, yang menjadi dasar untuk pengolahan teks di chapter berikutnya.